Assignment 6 – UVM - Soln

```
class ALSU_agent extends uvm_agent;
  `uvm_component_utils(ALSU_agent)
   ALSU_sequencer sqr;
   ALSU_driver drv;
   ALSU_monitor mon;
   ALSU_config ALSU_cfg;
   uvm_analysis_port #(ALSU_seq_item) agt_ap;

  function new (string name = "ALSU_agent", uvm_component parent = null);
   super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(ALSU_config)::get(this , "", "CFG", ALSU_cfg))begin
      `uvm_fatal("build_phase" , "Unable to get configuration object")
    end

    sqr = ALSU_sequencer::type_id::create("sqr",this);
    drv = ALSU_driver::type_id::create("drv",this);
    mon = ALSU_monitor::type_id::create("mon",this);
    agt_ap = new("agt_ap", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    drv.ALSU_vif = ALSU_cfg.ALSU_vif;
    mon.ALSU_vif = ALSU_cfg.ALSU_vif;
    drv.seq_item_port.connect(sqr.seq_item_export);
    mon.mon_ap.connect(agt_ap);
  endfunction
endclass
```

```
class ALSU_config extends uvm_object;
  `uvm_object_utils(ALSU_config)

  virtual ALSU_if ALSU_vif;

  function new(string name = "ALSU_config");
    super.new(name);
  endfunction
endclass
```

```
class ALSU_env extends uvm_env;
  `uvm_component_utils(ALSU_env)

  ALSU_agent agt;
  ALSU_scoreboard sb;
  ALSU_coverage cov;

  function new(string name = "ALSU_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agt = ALSU_agent::type_id::create("agt",this);
    sb = ALSU_scoreboard::type_id::create("sb",this);
    cov = ALSU_coverage::type_id::create("cov",this);
  endfunction: build_phase

  function void connect_phase(uvm_phase phase);
    agt.agt_ap.connect(sb.sb_export);
    agt.agt_ap.connect(cov.cov_export);
  endfunction
endclass
```

```
class ALSU_driver extends uvm_driver #(ALSU_seq_item);
  `uvm_component_utils(ALSU_driver)

  virtual ALSU_if ALSU_vif;
  ALSU_seq_item stim_seq_item;

  function new(string name = "ALSU_driver", uvm_component parent = null);
     super.new(name, parent);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      stim_seq_item = ALSU_seq_item::type_id::create("stim_seq_item");
      seq_item_port.get_next_item(stim_seq_item);
      ALSU_vif.direction = direction_e'(stim_seq_item.direction);
      ALSU_vif.cin = stim_seq_item.cin;
      ALSU_vif.red_op_A = stim_seq_item.red_op_A;
      ALSU_vif.red_op_B = stim_seq_item.red_op_B;
      ALSU_vif.bypass_A = stim_seq_item.bypass_A;
      ALSU_vif.bypass_B = stim_seq_item.bypass_B;
      ALSU_vif.opcode = opcode_e'(stim_seq_item.opcode);
      ALSU_vif.serial_in = stim_seq_item.serial_in;
      ALSU_vif.A = stim_seq_item.A;
      ALSU_vif.B = stim_seq_item.B;
      ALSU_vif.rst = stim_seq_item.rst;
      @(negedge ALSU_vif.clk);
      seq_item_port.item_done();
      `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
    end
  endtask

endclass
```

Add functional coverage done together in the following

```
class ALSU_coverage extends uvm_component;
`uvm_component_utils(ALSU_coverage)
uvm_analysis_export #(ALSU_seq_item) cov_export;
uvm_tlm_analysis_fifo #(ALSU_seq_item) cov_fifo;
ALSU_seq_item seq_item_cov;

covergroup cvr_grp;
   A_cvp : coverpoint seq_item_cov.A;
   B_cvp : coverpoint seq_item_cov.B;
   // Add your coverpoints
endgroup

 function new(string name = "ALSU_coverage", uvm_component parent = null) ;
   super.new(name, parent);
   cvr_grp = new();
 endfunction

function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  cov_export = new("cov_export", this);
  cov_fifo = new("cov_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
   super.connect_phase(phase);
   cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
  super.run_phase(phase);
  forever begin
    cov_fifo.get(seq_item_cov);
    cvr_grp.sample();
  end
endtask

endclass
```

```
class ALSU_main_sequence extends uvm_sequence #(ALSU_seq_item);
  `uvm_object_utils(ALSU_main_sequence);
   ALSU_seq_item seq_item;

  function new (string name = "ALSU_main_sequence");
    super.new(name);
  endfunction

  task body;
    repeat(10000) begin
        seq_item = ALSU_seq_item::type_id::create("seq_item");
        start_item(seq_item);
        assert(seq_item.randomize());
        finish_item(seq_item);
    end
  endtask
endclass
```

```
class ALSU_monitor extends uvm_monitor;
  `uvm_component_utils(ALSU_monitor)
  virtual ALSU_if ALSU_vif;
  ALSU_seq_item rsp_seq_item;
  uvm_analysis_port #(ALSU_seq_item) mon_ap;

 function new(string name = "ALSU_monitor", uvm_component parent = null) ;
   super.new(name, parent);
 endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
  endfunction: build_phase

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
      rsp_seq_item = ALSU_seq_item::type_id::create("rsp_seq_item");
      @(negedge ALSU_vif.clk);
      rsp_seq_item.direction = direction_e'(ALSU_vif.direction);
      rsp_seq_item.cin = ALSU_vif.cin;
      rsp_seq_item.red_op_A = ALSU_vif.red_op_A;
      rsp_seq_item.red_op_B = ALSU_vif.red_op_B;
      rsp_seq_item.bypass_A = ALSU_vif.bypass_A;
      rsp_seq_item.bypass_B = ALSU_vif.bypass_B;
      rsp_seq_item.opcode = opcode_e'(ALSU_vif.opcode);
      rsp_seq_item.serial_in = ALSU_vif.serial_in;
      rsp_seq_item.A = ALSU_vif.A;
      rsp_seq_item.B = ALSU_vif.B;
      rsp_seq_item.rst = ALSU_vif.rst;
      rsp_seq_item.out = ALSU_vif.out;
      rsp_seq_item.leds = ALSU_vif.leds;
      mon_ap.write(rsp_seq_item);
      `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
    end
  endtask
endclass
```

```
class ALSU_reset_sequence extends uvm_sequence #(ALSU_seq_item);
  `uvm_object_utils(ALSU_reset_sequence);
   ALSU_seq_item seq_item;

  function new (string name = "ALSU_reset_sequence");
    super.new(name);
  endfunction

  task body;
    seq_item = ALSU_seq_item::type_id::create("seq_item");
    start_item(seq_item);
    seq_item.cin = 0;
    seq_item.red_op_A = 0;
    seq_item.red_op_B = 0;
    seq_item.bypass_A = 0;
    seq_item.bypass_B = 0;
    seq_item.direction = direction_e'(0);
    seq_item.serial_in = 0;
    seq_item.opcode = opcode_e'(0);
    seq_item.A = 0;
    seq_item.B = 0;
    seq_item.rst = 1;
    finish_item(seq_item);
  endtask
endclass
```

```
class ALSU_scoreboard extends uvm_scoreboard;
`uvm_component_utils(ALSU_scoreboard)
uvm_analysis_export #(ALSU_seq_item) sb_export;
uvm_tlm_analysis_fifo #(ALSU_seq_item) sb_fifo;
ALSU_seq_item seq_item_sb;
logic [15:0] leds_ref;
logic [5:0] out_ref;
logic cin_reg, red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, serial_in_reg;
direction_e direction_reg;
opcode_e opcode_reg;
logic [2:0] A_reg, B_reg;

int error_count = 0;
int correct_count = 0;

 function new(string name = "ALSU_scoreboard", uvm_component parent = null) ;
   super.new(name, parent);
 endfunction

function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  sb_export = new("sb_export", this);
  sb_fifo = new("sb_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
   super.connect_phase(phase);
   sb_export.connect(sb_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
  super.run_phase(phase);
  forever begin
    sb_fifo.get(seq_item_sb);
    ref_model(seq_item_sb);
    if (seq_item_sb.out !== out_ref || seq_item_sb.leds !== leds_ref) begin
      `uvm_error("run_phase", $sformatf("Comparsion failed, Transaction received by the DUT:%s  While the r
      error_count++;
    end
    else begin
      `uvm_info("run_phase", $sformatf("Correct ALSU out: %s ", seq_item_sb.convert2string()), UVM_HIGH);
      correct_count++;
    end
  end
endtask
```

Add the constraints we have done together in the following

```
class ALSU_seq_item extends uvm_sequence_item;
 `uvm_object_utils(ALSU_seq_item)

  rand bit clk, rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, serial_in;
  rand direction_e direction;
  rand opcode_e opcode;
  rand bit signed [2:0] A, B;
  logic [15:0] leds;
  logic [5:0] out;

  function new(string name = "ALSU_seq_item");
  super.new(name);
  endfunction

  function string convert2string();
    return $sformatf("%s reset = 0b%b, cin = 0b%b, red_op_A = 0b%b, red_op_B = 0b%b, bypass_A = 0b%b,
      bypass_B = 0b%b,  serial_in = 0b%b, direction = %s,  opcode = %s, A = 0b%b, B = 0b%b, leds = 0b%b, out = 0b%b",
      super.convert2string(), rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, serial_in, direction, opcode, A, B, leds, out);
  endfunction

  function string convert2string_stimulus();
    return $sformatf("%s reset = 0b%b, cin = 0b%b, red_op_A = 0b%b, red_op_B = 0b%b, bypass_A = 0b%b,
      bypass_B = 0b%b,  serial_in = 0b%b, direction = %s,  opcode = %s, A = 0b%b, B = 0b%b",
      super.convert2string(), rst, cin, red_op_A, red_op_B, bypass_A, bypass_B, serial_in, direction, opcode, A, B);
  endfunction


  constraint rst_con {
  rst dist {1:= 2, 0:= 98};
  }

  //Add your constraints
endclass
```

```
class ALSU_sequencer extends uvm_sequencer #(ALSU_seq_item);
  `uvm_component_utils(ALSU_sequencer);

  function new (string name = "ALSU_sequencer", uvm_component parent = null);
    super.new(name, parent);
  endfunction
endclass
```

```
package shared_pkg;
typedef enum {RIGHT, LEFT} direction_e;
typedef enum bit [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;
endpackage
```

```
class ALSU_test extends uvm_test;
  `uvm_component_utils(ALSU_test)

  ALSU_env env;
  ALSU_config ALSU_cfg;
  virtual ALSU_if ALSU_vif;
  ALSU_main_sequence main_seq;
  ALSU_reset_sequence reset_seq;

  function new(string name = "ALSU_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = ALSU_env::type_id::create("env",this);
    ALSU_cfg = ALSU_config::type_id::create("ALSU_cfg",this);
    main_seq = ALSU_main_sequence::type_id::create("main_seq",this);
    reset_seq = ALSU_reset_sequence::type_id::create("reset_seq",this);

    if (!uvm_config_db #(virtual ALSU_if)::get(this, "","ALSU_IF", ALSU_cfg.ALSU_vif))
      `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the ALSU from the uvm_config_db");

    uvm_config_db #(ALSU_config)::set(this, "*", "CFG", ALSU_cfg);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_objection(this);
    //reset sequence
    `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
    reset_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Reset Deasserted", UVM_LOW)

    //main sequence
    `uvm_info("run_phase", "Stimulus Generation Started", UVM_LOW)
    main_seq.start(env.agt.sqr);
    `uvm_info("run_phase", "Stimulus Generation Ended", UVM_LOW)
    phase.drop_objection(this);
  endtask
endclass: ALSU_test
```

```
module top();
  bit clk;

  initial begin
    forever
      #1 clk = ~clk;
  end

  ALSU_if ALSUif (clk);
  ALSU DUT (ALSUif.A, ALSUif.B, ALSUif.cin, ALSUif.serial_in, ALSUif.red_op_A, ALSUif.

  initial begin
    uvm_config_db#(virtual ALSU_if)::set(null, "uvm_test_top", "ALSU_IF", ALSUif);
    run_test("ALSU_test");
  end

endmodule
```

Kareem Waseem